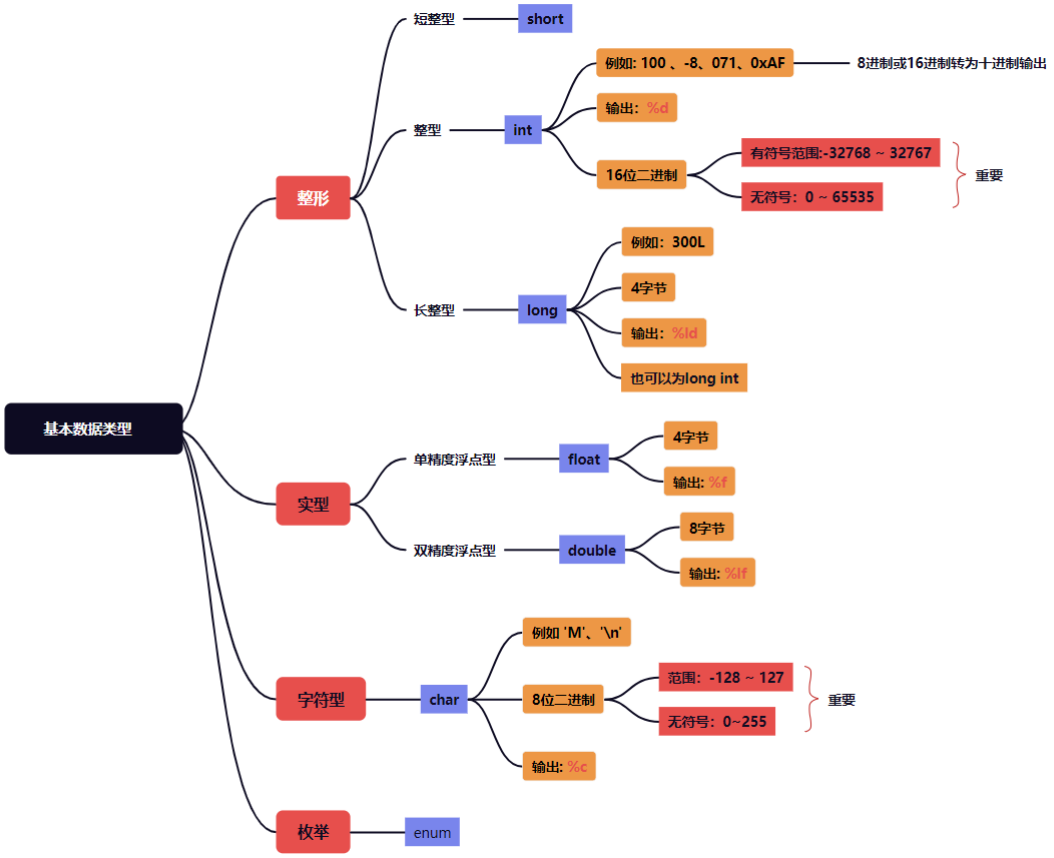
C语言的常量、基本数据类型、运算符

一、标识符(变量、常量、函数、数组)命名规则

- 组成：字母、数字、下划线，不能以数字开头
- 区分大小写
- 不能与关键字重名。 int if = 800; X
  - C语言一共提供了32个关键字，34个运算符

二、基本数据类型



- 整型和字符型可以混合计算

```
char k='y';
k-=32;
```

```
putchar(k); // Y
```

二、常量

- **整形常量 (3种表示形式)： 不提供二进制常量**
  - **十进制整数 ：**
    - **2、-983**
    - **输出 %d**
  - **八进制整数：**
    - **以 `0` 开头， 由0~7 构成; 如 012 表示八进制整数12**
    - **输出 `%o`**
  - **十六进制整数**
    - **以`0x`开头，由 0~9、A~F或a~f构成**
    - **如0x12A 表示16进制整数12A**
    - **输出 `%x 或%X`**

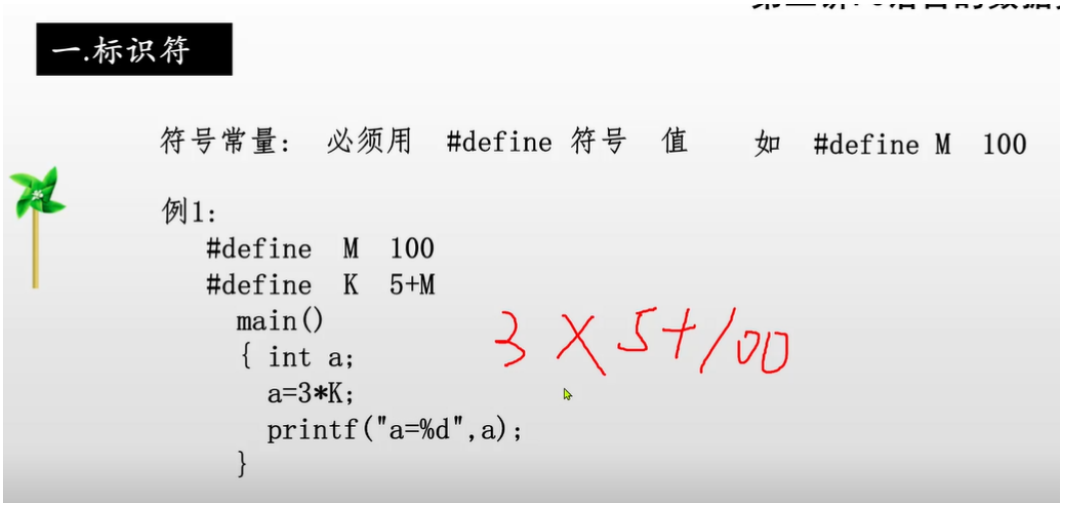
```
int k = 0101;
pritf("k=%x\n",k) ==>  41 //8进制转16进制
```

- **实型常量**
  - **单精度 ：**
    - **9.76f 、5.6f ;**
    - **float k =263; float kk = 9.76;**
  - **双精度: 3.25、9.7**
- **字符常量 ：**
  - **'M' 'y' '9' '\n' '\r'、'\\'**
  - **字符常量的值 : 该字符的ASCII值 ：**

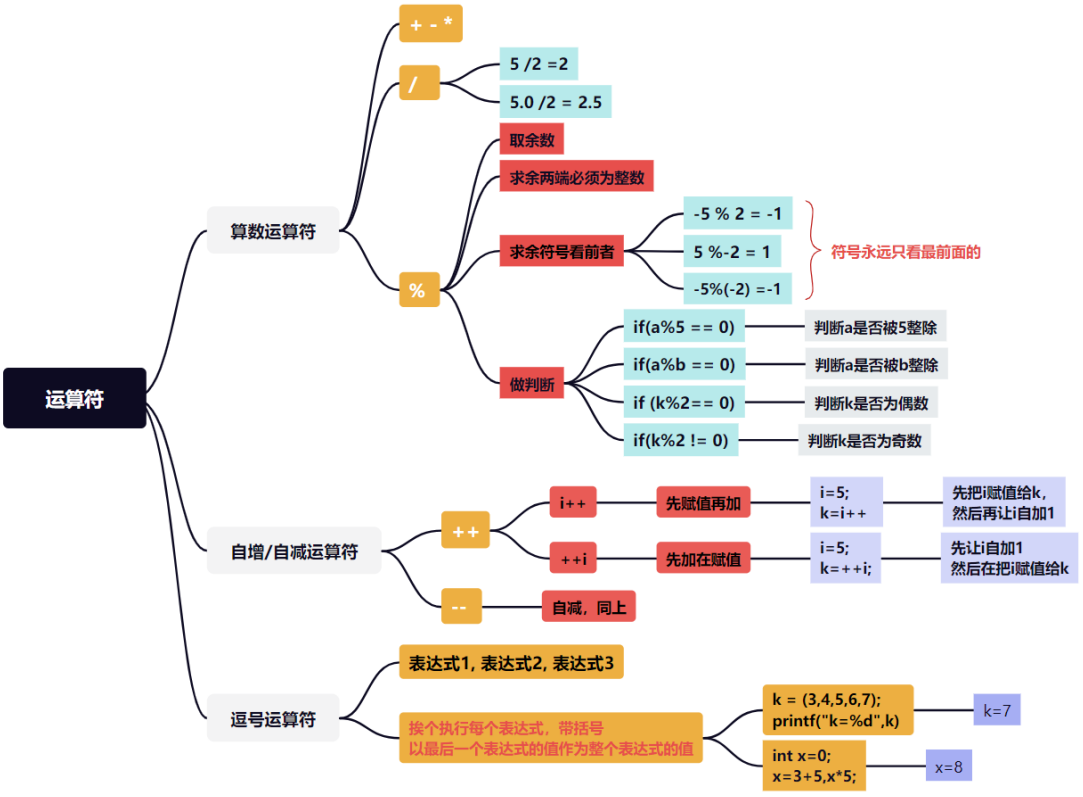
```
以下方式定义等效
char a =65;
char b = 'A';
```

```
char c = '\101';
char d = 0x41;
char e = 0101;
-----------------------------
printf("%c\n", '\141');  // \141表示ASCII值为八进制141的字
-----------------------------
char k = 97;  printf("%c",k) //输出ASCII对应的字符a
char h = '\018'; // X;8进制由0~7组成
```

- **字符串常量 ：（用双引号引起来）**
  - **“M”,“windows”“中国”**
  - **每个字符串结尾自动加一个 ”\0” 作为字符串结束标志**
  - **C语言不提供字符串变量，只提供字符数组**
    - **char k[下标] = “windows”; 赋值可以省下标，没有赋值不能省下标**
    - **char k[] = {'a','b'};**
  - **printf(”%s”,k) ：使用%s输出字符串**
- **科学计数法常量 ：**
  - **2e3 3e-5 -2.67E4**
  - **e前必有数，e后必整数**
  - **1.3e-1 → 1.3 * 10的-1次方**
- **符号常量**



三、运算符



- **整形/整形 一定是整形**
- **%的特殊用法**

```
//如何取出千/百/十/个位？
千： 数/1000%10
百： 数/100%10
十： 数/10%10
个： 数%10
```

- **自增的用法**
  - **自增或自减运算符只能用于变量，不能用于符号常量**
    - **#define E 2； E++; →(X)**

```
#include <stdio.h>
void main()
{
    int k,m=5;
    k= m++;
```

```
    k= ++m;
    k= m++;
    k= --m;  // m=7; k=7; 从倒数第二行开始算，前面m加了3次，m = 8
    k= ++m;
    printf("k=%d,m=%d\n",k,m); // k=8,m=8
}
---------------------------------------
#include <stdio.h>
int main()
{
   int m =7;
   if(m++>7)  // 先进行判断，m>7，之后执行m++,此时m=8
        printf("%d\n", m++);
    else
        printf("%d\n", --m); //7
}
---------------------------------------
double x= 3.4,y=3.2;
y+=x++;
printf("%lf",y); //6.6; 整个表达式走完后，x才加1

---------------
char c='A';
c=c+++1; // c++ +1;c++是一个整体，先取出c，进行c+1 赋值给c；  c=c++
printf("%c",c); // 'B'
------------------------
 int x,y,z;
 x=y=1;
 z=x++,y=x++; // 逗号相当于一句话完成
 printf("%d,%d,%d",x,y,z); //3,2,1
```

- **关系运算符**

关系运算符	名称	示例	优先级	
>	大于	a>3	相同	高
>=	大于或等于	a>=3		
<=	小于或等于	a<=b-c		
<	小于	y+x>3*sin(x)		
= =	等于	i==5	相同	低
!=	不等于	m!=n		

- **关系表达式成立时，其值为1(真)**
- **关系表达式不成立时，其值为0(假)**

- **逻辑运算符**

逻辑运算符	名称	示例	优先级
!	逻辑非	!(x>9)	高
&&	逻辑与	(x>2) &&(y<9)	
||	逻辑或	(x>2) ||(y<9)	低

- **当运算对象为0时，即判定其为假，当运算对象为非0的任何值(包括负值)，即判定为真**

```
a=2,b=0;
则：!a 等于0。  !b等于1
a&&b 等于0
a||b 等于1
```

- **&& ：只要有一个运算对象为假，剩余的不再执行。 短路**
  - **例如: a&&b&&c ,如果a为假，b和c不执行**
- **|| ：只要有一个表达式的值为真，结果就是真，剩余不执行; 短路**
  - **a || b || c,如果a为真，b和c不执行**
- **条件运算符**

- 条件 ？表达式1 ：表达式 2
- 例如： 找出a,b的最大值 ： k = a>b ? a : b;

- 赋值运算符
  - 普通赋值 ： =
    - int h = 6; int k ,m=5; → k=0;
    - 注意： 定义变量时不能连等赋值（ int a=b=c=10; X)
    - 先定义再连等：int a ,b,c; a=b=c=10;
    - 定义时分开赋值：int a =100,b=20,c=30;
    - 等号左边必须是变量: a=c+2=b; X (c+2错误)
  - 复合赋值

    +=　-=　*=　/=　%=

    - 如： x+=3;　→　x =x +3 ;
    - 复合赋值右侧表达式看为一个整体
      - a=5,b=3,k=7; k*=a+b; → 等价于 k = k*(a+b);

- 逗号运算符
  - k=(3,4,5,6,7) → k=7
  - k=3,4,5,6,7 → k=3
  - k=3,k=4,k=5,k=6,k=7 →k=7

- 位运算符
  - 位运算符的优先级从高到低，依次为~、&、^、|
    - ~ (按位取反）0变1，1变0
    - &(与):　见0 为0，全1为1;
    - |(或)：　见1 为 1，全 0 为 0
    - ^(异或)　相同为0 ，不同为1

- 强制类型转换
  - （类型名）表达式
  - 强制类型转换，只转离它最近的表达式

```c
#include <stdio.h>
void main()
{
    float a=5.7, b=8.6; int k;
    k= (int) a+b;   // 5+8.6 --> 13
    printf("k=%d\n", k);  // k=13
}
```

- **对原有的数据类型不破坏**

表0-8　运算符优先级

括号	逻辑非	算术运算符	关系运算符	逻辑与	逻辑或	条件运算符	赋值运算符
()	!	+、-、*、/、%、++、- -	>、<、>=、<=、==、!=	&&	||	? :	=、复合赋值运算符
高	←						低

- **算> 关>逻>条>赋>逗**

易错题：　a=5 b=3

k=(a=6) || (b=7)	k=1 a=6 b=3
k=(a=6 || b==7)	k=1 a=6 b=3
k=(a==6) || (b=7)	k=1 a=5 b=7
k=(a==6) || (b==7)	k=0 a=5 b=3
k=(a=6)&&(b=7)	k=1 a=6 b=7
k=(a=6)&&(b==7)	k=0 a=6 b=3
k=(a==6)&&(b=7)	k=0 a=5 b=3
k=(a==6)&&(b==7)	k=0 a=5 b=3

四、转义字符 （第10页）

以`\`开头的字符序列，为转义字符

转义字符	输出
\n	回车换行
\b	退格
\t	跳格，跳过一个table键
\\	反斜杠.输出一个\
\'	单引号'
\"	双引号"
\ddd	1~3位八进制数转为ASCII的字符
\xhh	1~2位16进制数转为ascII的字符

```
// 八进制141转为 10进制为97，对应的ASCII为a
   printf("%c\n",'\141');
   // 10进制为97
   printf("%d\n",'\141');
   // 16进制转为10进制输出 =
   printf("%c\n",'\x3d');
```

五、数据交换

- **a和b的互换**

```
t =a;
a =b;
b =t;
```

```
//数字类型
x=x+y;
y=x-y;
x=x-y;
```

- **保留2位小数的四舍五入**

```
float k =23.567;
k*=1000;                  // 23567
k=(int)((k+5)/10);        //23572 -->2357.2 -->2357
k/=100;
printf("%f",k); //23.57
```

- **保留三位小数的四舍五入**

```
float k =23.5676;
k*=10000;
k=(int)((k+5)/10);
k/=1000;
printf("%f",k); //23.568
```

六、错题集

- **用int接收float类型，只保留整数**

```
int c1=1,c2=2,c3;
c3=1.0/c2*c1;
printf("%d\n",c3);  // 0
--------------------------------
int c4=2.56+7;
printf("%d",c4); //9
```

- **下列变量中定义合法的是**
  - **short _a = 13e-1; → short _a= 0; (X)，影响数据准确性**
  - **double b=1+5e2.5; 5e2.5 (X)，e后面不能是小数**
  - **long d0= 0xfdaL; 正确**
- **整型与整型永远是整形**
  - **9/2 =4.5 (X)**
  - **9/2 =4 √**
  - **856/100=8 √**
- **赋值语句等号左边必须是无操作的变量**
  - **int a =10,b; a=a+1=b (X). 红色标注为错误部分**
- **重要: 反运算**

```
int k = ~-36 ^ 36;
先把-36转为补码，再进行取反操作
```

```
int k = ~36 ^ 35;
```
正数先取反，在进行异或操作，最后转为补码

下列整型常数表示正确的是____。

A. 123L

B. 123u

C. 0x123

D. 0123

A.123L-这是一个长整型数，以'L'结尾。这是正确的表示方法。B.123u-这是一个无符号整型数，以'u'结尾。这也是正确的表示方法。C.0x123-这是一个十六进制整型数，以'0x'开头。这是正确的表示方法。D.0123-这是一个八进制整型数，以'0'开头。这也是正确的表示方法。

所以，这四个选项都是正确的。

解释：

A选项是一个长整型数，以'L'结尾。在C语言中，长整型数可以用'L'或'l'结尾，例如：235L、1000l等。

B选项是一个无符号整型数，以'u'结尾。无符号整型数用于表示正整数，以'u'或'U'结尾，例如：300u、65535U等。

C选项是一个十六进制整型数，以'0x'开头。十六进制数以'0x'或'0X'开头，然后是数字0-9和字母A-F（或a-f）组成的数值，例如：0x123、0XAb等。

D选项是一个八进制整型数，以'0'开头。八进制数以数字0开头，然后是数字0-7组成的数值，例如：0123、076等。

- **以下符合正确的c语言语法的赋值表达式是 BD**

  A: d=9+e+f=d+3;　B：d=9+e,f=d+3;

  C: c!=a+b;　　D：a+=(b=5)*(c=3)

  - **赋值表达式，等号左边必须是变量，不能是常量或算数表达式,所以A错**
  - **!= 不是赋值运算符，是关系运算符。C错误**
  - **int a= 1,b=2,c=3;**
    a+=(b=5)*(c=3); //输出结果是16
- **int x=3,y=4,z=5, 求以下表达式的值;**
  - **x||++y&&y-z**
    - **先执行x，x!=0; 输出1，后面部分不执行.**
  - **!(x<y&&!z||1)**
    - **这种复杂表达式，看到或，先看最简单部分, 1!=0,输出1，最外面取反，输出0**